CO4- LAB EXPERIMENTS

1. Create a Web Blog using WordPress – Anchor Tag, Title Tag, etc.

Code

```
<!DOCTYPE html>
<html>
<head>
   <title>My Technology Blog</title>
</head>
<body>

   <h1>Welcome to My Technology Blog</h1>

   <h2>About Technology</h2>
   <p>This blog contains information about modern technology.</p>

   <a href="https://www.google.com">Visit Google</a>
   <br><br>

   <a href="#about">Go to About Section</a>

   <h2 id="about">About</h2>
   <p>This section describes the purpose of the blog.</p>

   <img src="technology.jpg" alt="Technology Image" width="300">

</body>
</html>
```

Input

Create a WordPress blog page with:

Title Tag

Heading Tags

Anchor Tag

Image Tag

Paragraph Tag

Output

My Technology Blog

Welcome to My Technology Blog

About Technology

This blog contains information about modern technology.

Visit Google

Go to About Section

About

This section describes the purpose of the blog.

---

2. Basic Linear Regression

Code

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])

model = LinearRegression()
```

```
model.fit(X, y)

x = float(input("Enter value of X: "))
prediction = model.predict([[x]])

print("Predicted value:", prediction[0])
```

Input

Enter value of X: 6

Output

Predicted value: 12.0

---

3. K-Nearest Neighbors (KNN) Classification

Code

```
from sklearn.neighbors import KNeighborsClassifier

X = [[1, 1], [2, 2], [3, 3], [6, 6], [7, 7], [8, 8]]
y = ["A", "A", "A", "B", "B", "B"]

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

x1 = float(input("Enter first value: "))
x2 = float(input("Enter second value: "))

prediction = model.predict([[x1, x2]])

print("Predicted Class:", prediction[0])
```

Input

Enter first value: 2

Enter second value: 3

Output

Predicted Class: A

---

4. Neural Network using TensorFlow for Image Classification

Code

```
import tensorflow as tf
import numpy as np

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.fit(x_train, y_train, epochs=3, verbose=1)

loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
```

print("Test Accuracy:", round(accuracy * 100, 2), "%")

Input

MNIST image dataset is automatically loaded.

Output

Epoch 1/3

...

Epoch 2/3

...

Epoch 3/3

...

Test Accuracy: 97.00 %

---

5. Decision Tree Classification

Code

```
from sklearn.tree import DecisionTreeClassifier

X = [[1], [2], [3], [4], [5], [6]]
y = ["No", "No", "No", "Yes", "Yes", "Yes"]

model = DecisionTreeClassifier(random_state=1)
model.fit(X, y)

age = int(input("Enter age: "))

prediction = model.predict([[age]])

print("Prediction:", prediction[0])
```

Input

Enter age: 5

Output

Prediction: Yes

---

6. Sentiment Analysis using NLP

Code

```
from textblob import TextBlob

text = input("Enter a sentence: ")

polarity = TextBlob(text).sentiment.polarity

if polarity > 0:
    print("Sentiment: Positive")
elif polarity < 0:
    print("Sentiment: Negative")
else:
    print("Sentiment: Neutral")
```

Input

Enter a sentence: I really enjoyed this movie

Output

Sentiment: Positive

---

7. Genetic Algorithm for Mathematical Function

Code

```
import random

def fitness(x):
    return x * x
```

```
population = [random.randint(0, 31) for _ in range(10)]

for generation in range(10):
    population.sort(key=fitness, reverse=True)

    parents = population[:2]

    new_population = parents[:]

    while len(new_population) < 10:
        child = (parents[0] + parents[1]) // 2

        if random.random() < 0.1:
            child = random.randint(0, 31)

        new_population.append(child)

    population = new_population

best = max(population, key=fitness)

print("Best Solution:", best)
print("Maximum Value:", fitness(best))
```

Input

No input required.

Output

Best Solution: 31

Maximum Value: 961

---

8. Support Vector Machine (SVM) for Binary Classification

Code

```
from sklearn.svm import SVC

X = [[1, 1], [2, 2], [3, 3], [6, 6], [7, 7], [8, 8]]
y = [0, 0, 0, 1, 1, 1]

model = SVC(kernel="linear")
model.fit(X, y)

x1 = float(input("Enter first value: "))
x2 = float(input("Enter second value: "))

prediction = model.predict([[x1, x2]])

print("Predicted Class:", prediction[0])
```

Input

Enter first value: 7

Enter second value: 6

Output

Predicted Class: 1

---

9. Dimensionality Reduction using PCA

Code

```
from sklearn.decomposition import PCA
import numpy as np

X = np.array([
```

```
    [2, 4, 6],
    [3, 6, 9],
    [4, 8, 12],
    [5, 10, 15],
    [6, 12, 18]
])

pca = PCA(n_components=2)

X_reduced = pca.fit_transform(X)

print("Original Data:")
print(X)

print("\nReduced Data:")
print(X_reduced)
```

Input

No input required.

Output

```
Original Data:
[[ 2  4  6]
 [ 3  6  9]
 [ 4  8 12]
 [ 5 10 15]
 [ 6 12 18]]

Reduced Data:
[[-3.67423461e+00  0.00000000e+00]
 [-1.83711731e+00  0.00000000e+00]
```

[0.00000000e+00 0.00000000e+00]

[1.83711731e+00 0.00000000e+00]

[3.67423461e+00 0.00000000e+00]]

---

10. Basic Recommendation System using Collaborative Filtering

Code

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

data = {
    "Movie1": [5, 4, 0, 0],
    "Movie2": [4, 5, 0, 0],
    "Movie3": [0, 0, 5, 4],
    "Movie4": [0, 0, 4, 5]
}

ratings = pd.DataFrame(
    data,
    index=["User1", "User2", "User3", "User4"]
)

similarity = cosine_similarity(ratings)

print("User Similarity Matrix:")
print(similarity)

user = int(input("Enter user number (1-4): ")) - 1

similar_users = similarity[user].argsort()[::-1]
```

```
for u in similar_users:
    if u != user:
        print("Recommended Movies based on similar user:",
              list(ratings.columns[ratings.iloc[u] > 0]))
        break
```

Input

Enter user number (1-4): 1

Output

User Similarity Matrix:

```
[[1.         0.97560976 0.         0.        ]
 [0.97560976 1.         0.         0.        ]
 [0.         0.         1.         0.97560976]
 [0.         0.         0.97560976 1.        ]]
```

Recommended Movies based on similar user:

['Movie1', 'Movie2']